



PROJETO E ANÁLISE DE ALGORITMOS

AULA 7 – HEAP SORT E FILAS DE PRIORIDADE

7.1 INTRODUÇÃO

- Como a ordenação por inserção, mas diferentemente da ordenação por intercalação, a ordenação por heap **ordena no lugar** (in-place): apenas um número constante de elementos do vetor é armazenado fora do vetor de entrada em qualquer instante.
- Assim, a ordenação por heap **combina os melhores atributos** dos dois algoritmos de ordenação que já discutimos.

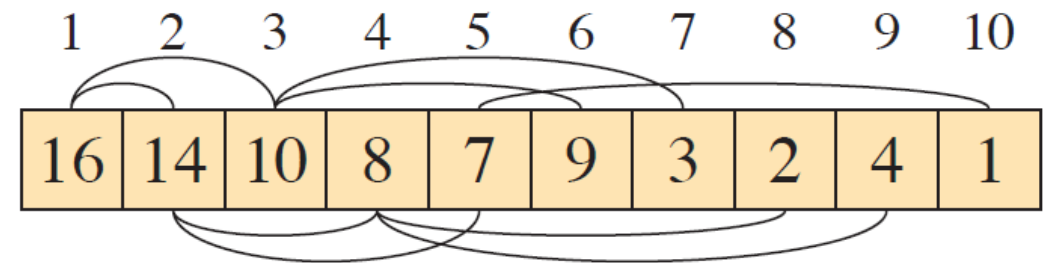
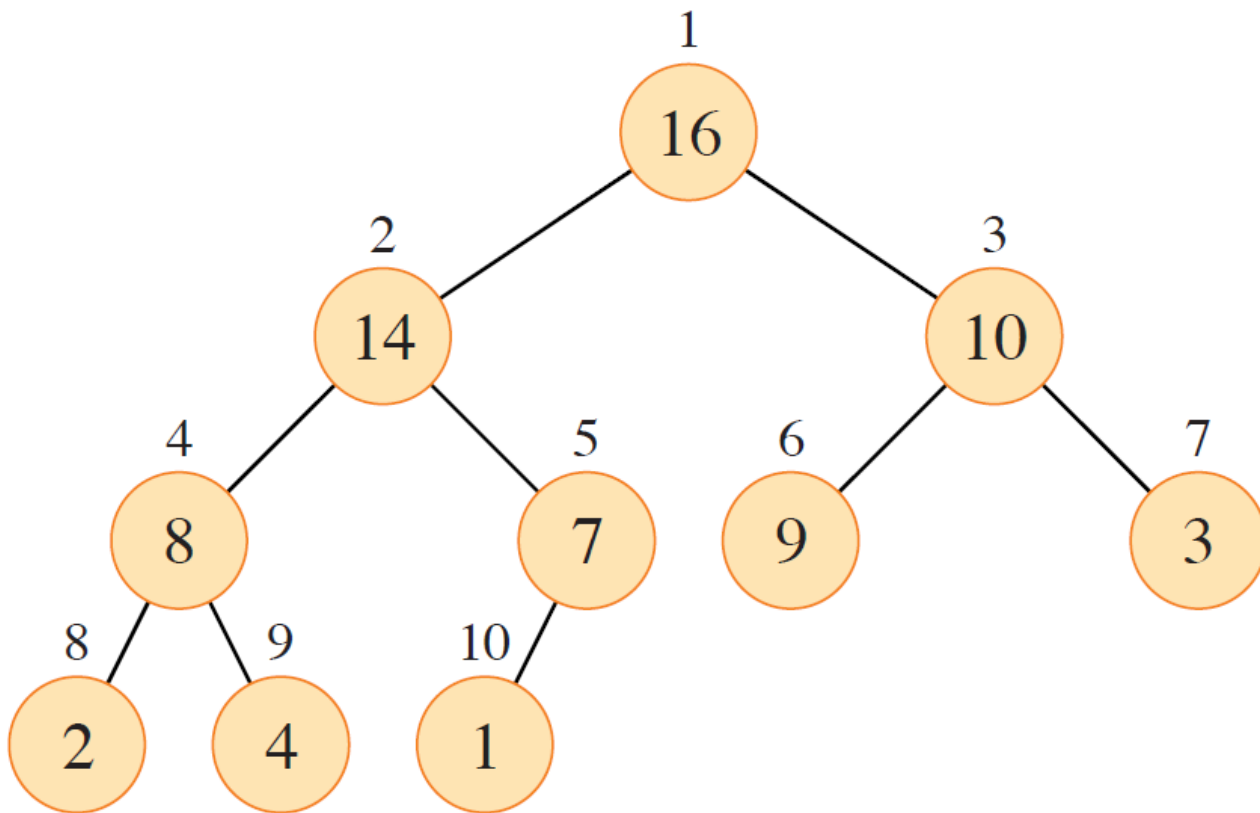
7.1 INTRODUÇÃO

- A ordenação por heap também introduz outra técnica de projeto de algoritmos: a utilização de uma estrutura de dados, nesse caso uma estrutura que denominamos “heap” para gerenciar informações. A **estrutura de dados heap** não é útil apenas para a ordenação por heap, ela também cria uma eficiente **fila de prioridades**.
- O termo “heap” foi cunhado originalmente no contexto da ordenação por heap, mas desde então passou a se referir também a “armazenamento com coleta de lixo”, tal como dado pelas linguagens de programação Java e Python. A estrutura de dados heap **não** é armazenamento com coleta de lixo.

7.2 HEAPS

- A estrutura de dados **heap (binário)** é um objeto do tipo vetor que pode ser visto como uma **árvore binária quase completa**.
- Cada nó da árvore corresponde a um elemento do vetor. A árvore está completamente preenchida em todos os níveis, exceto possivelmente no nível mais baixo, que é preenchido a partir da esquerda até um ponto.
- Um vetor $A[1 : n]$ que representa um heap é um objeto com um atributo `A.tamanho-do-heap`, que representa quantos elementos no heap estão armazenados dentro do vetor `A`.

7.2 HEAPS



7.2 HEAPS

- Embora $A[1 : n]$ possa conter números, somente os elementos em $A[1 : A.\text{tamanho-do-heap}]$, em que $0 \leq A.\text{tamanho-do-heap} \leq n$, são elementos válidos do heap.
- Se $A.\text{tamanho-do-heap} = 0$, então o heap está vazio.
- **A raiz da árvore é $A[1]$** e, dado o índice i de um nó, podemos calcular facilmente os índices de seu pai, do filho à esquerda e do filho à direita com os procedimentos de única linha PAI, ESQUERDA e DIREITA.

7.2 HEAPS

PAI(i)

return $\lfloor i/2 \rfloor$

ESQUERDA(i)

return $2i$

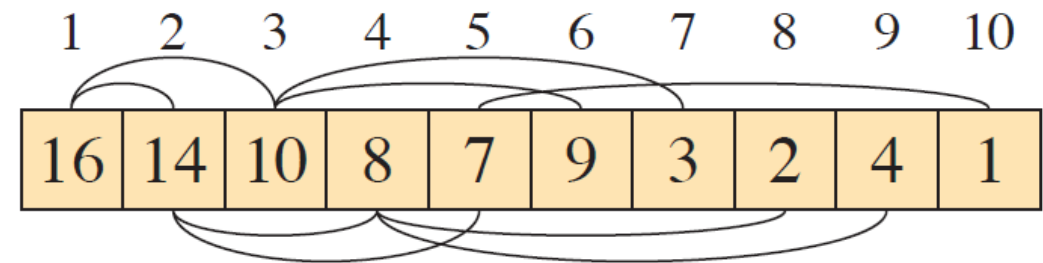
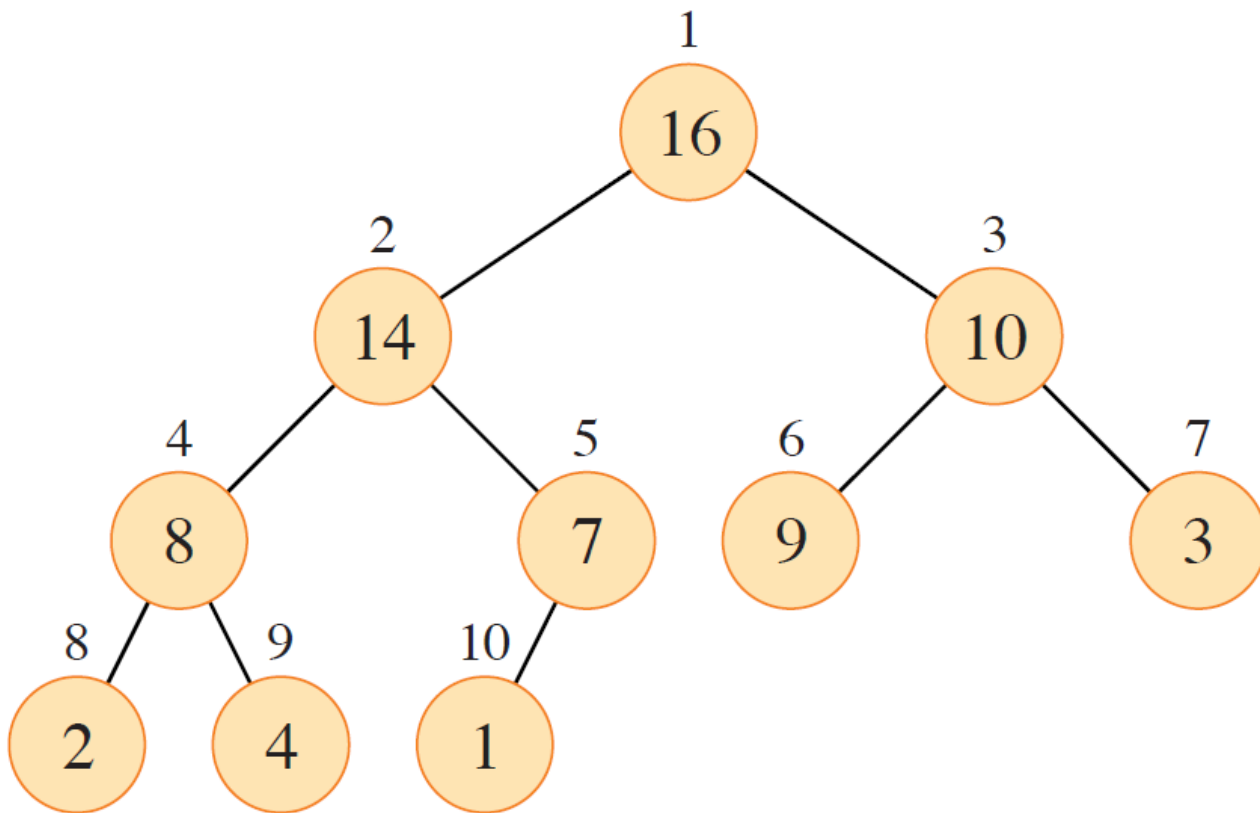
DIREITA(i)

return $2i + 1$

7.2 HEAPS

- Existem dois tipos de heaps binários: **heaps de máximo** e **heaps de mínimo**. Em ambos os tipos, os valores nos nós satisfazem a uma propriedade de heap, cujos detalhes específicos dependem do tipo de heap.
- Em um heap de máximo, a propriedade de heap de máximo é que, para todo nó i exceto a raiz, $A[\text{PAI}(i)] \geq A[i]$, isto é, o valor de um nó é, no máximo, o valor de seu pai. Assim, o maior elemento em um heap de máximo é armazenado na raiz, e a subárvore que tem raiz em um nó contém valores menores que o próprio nó.

7.2 HEAPS



7.2 HEAPS

- Um heap de mínimo é organizado de modo oposto; a propriedade de heap de mínimo é que, para todo nó i exceto a raiz, $A[PAI(i)] \leq A[i]$. O menor elemento em um heap de mínimo está na raiz. Para o algoritmo de ordenação por heap, usamos heaps de máximo. Heaps de mínimo são comumente empregados em filas de prioridades.
- Visualizando um heap como uma árvore, definimos a **altura** de um nó em um heap como o número de arestas no caminho descendente simples mais longo desde o nó até uma folha e definiremos a altura do heap como a altura de sua raiz.

7.2 HEAPS

- Visto que um heap de n elementos é baseado em uma árvore binária completa, sua altura é $\Theta(\lg n)$. As operações básicas em heaps são executadas em tempo que é, no máximo, proporcional à altura da árvore e, assim, demoram um tempo $O(\lg n)$.

7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP

- Para **mantermos a propriedade de heap** de máximo, chamamos o procedimento MAXIMIZA-HEAP. Suas entradas são um vetor A com o atributo tamanho-do-heap e um índice i para o vetor.
- Quando chamado, MAXIMIZA-HEAP **considera** que as árvores binárias com raízes em ESQUERDA(i) e DIREITA(i) são heaps de máximo, mas que $A[i]$ pode ser menor que seus filhos, o que **viola a propriedade de heap** de máximo.
- MAXIMIZA-HEAP permite que o valor em $A[i]$ “**flutue para baixo**” no heap de máximo, de modo que a subárvore com raiz no índice i obedeça à propriedade do heap de máximo.

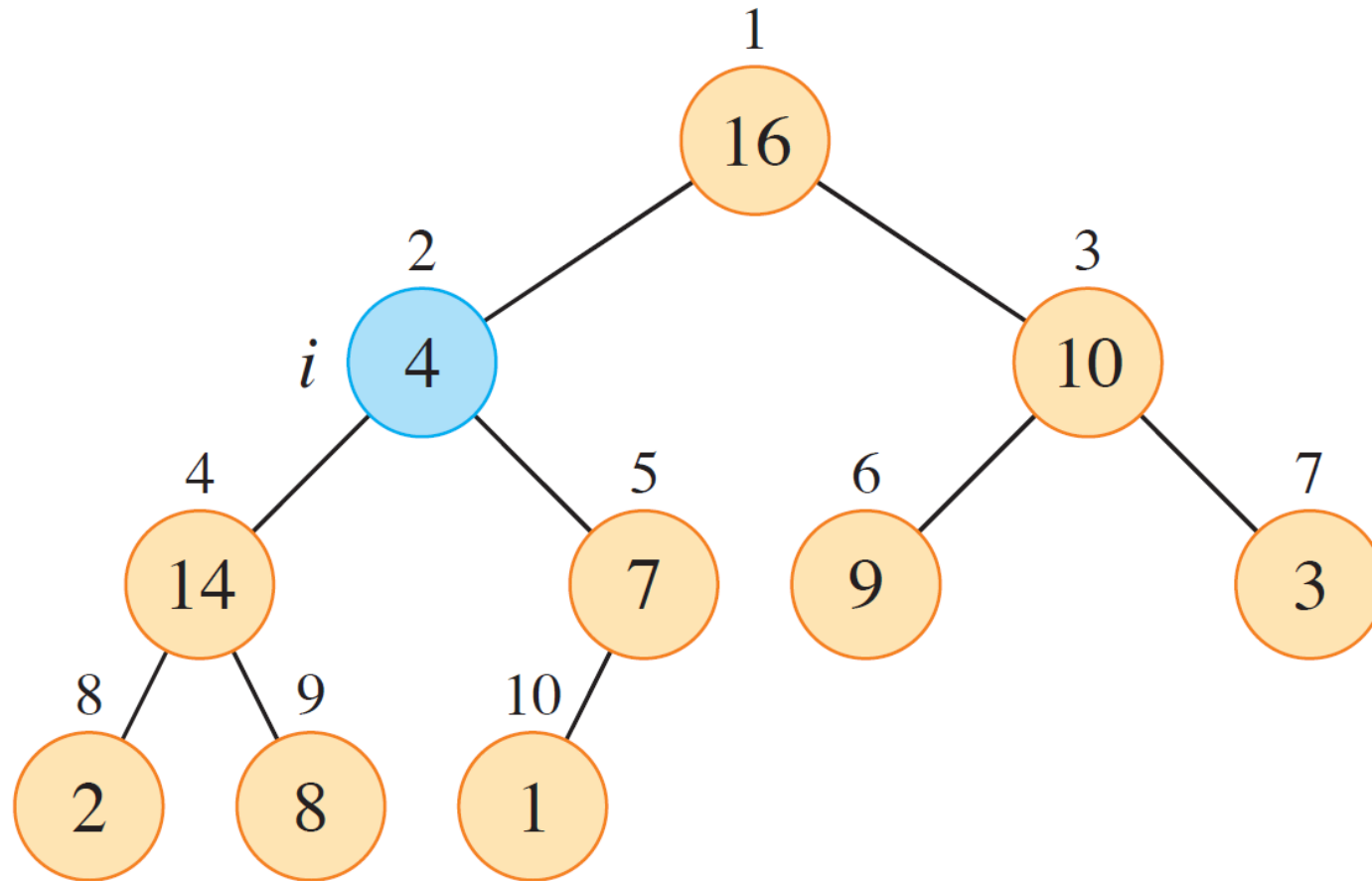
7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP

- Em cada etapa do procedimento, o **maior** dos elementos $A[i]$, $A[\text{ESQUERDA}(i)]$ e $A[\text{DIREITA}(i)]$ é determinado, e seu índice é armazenado em maior.
- Se $A[i]$ é maior, a subárvore com raiz no nó i já é um heap de máximo, e nada mais precisa ser feito.
- Caso contrário, um dos dois filhos tem o maior elemento.

7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP

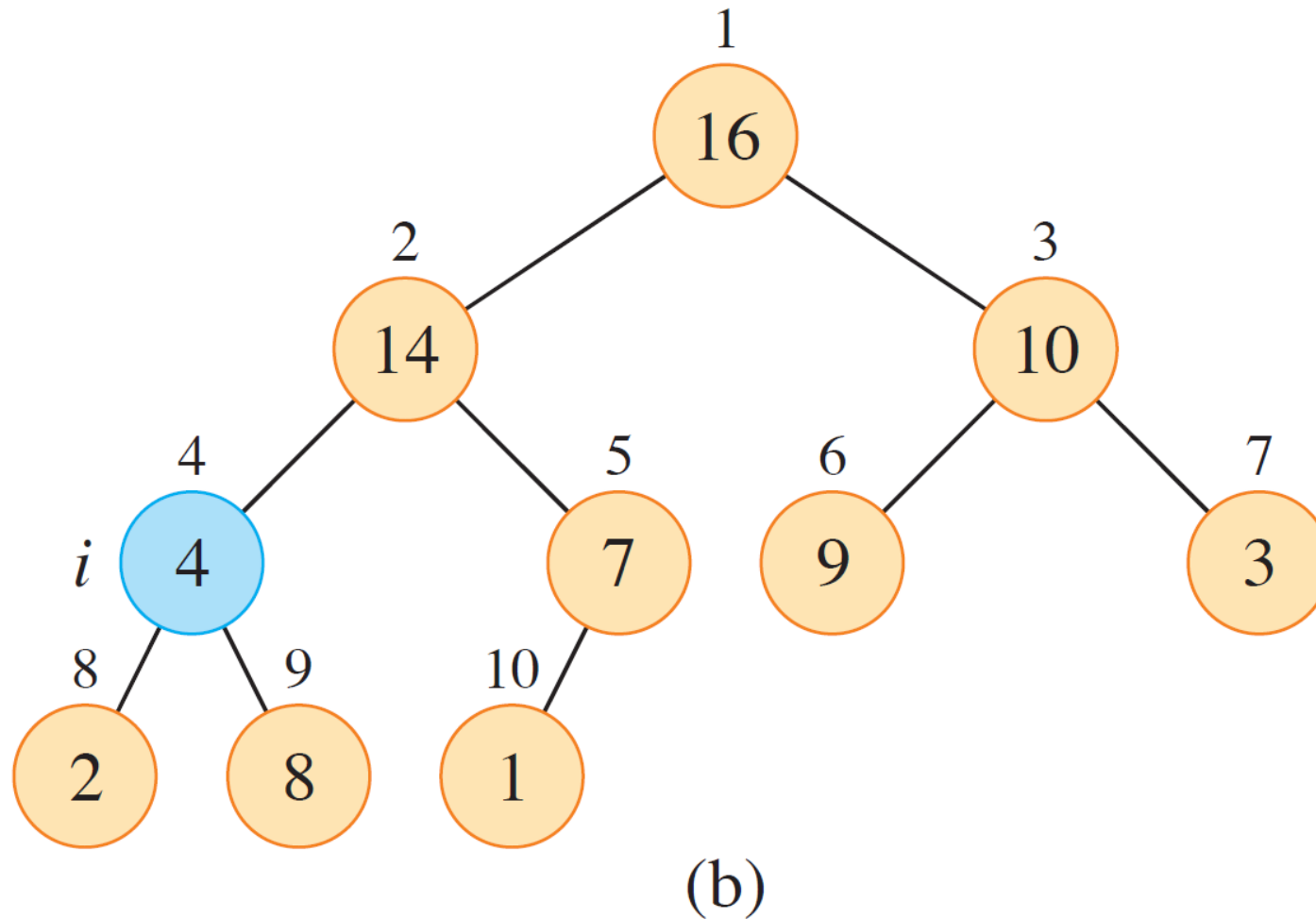
- As posições i e $maior$ trocam seu conteúdo, fazendo com que o nó i e seus filhos satisfaçam à propriedade de heap de máximo.
- Porém, agora o nó indexado por $maior$ tem seu valor diminuído e, assim, a subárvore com raiz em $maior$ poderia violar a propriedade de heap de máximo. Em consequência disso, chamamos MAXIMIZA-HEAP **recursivamente nessa subárvore**.

7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP

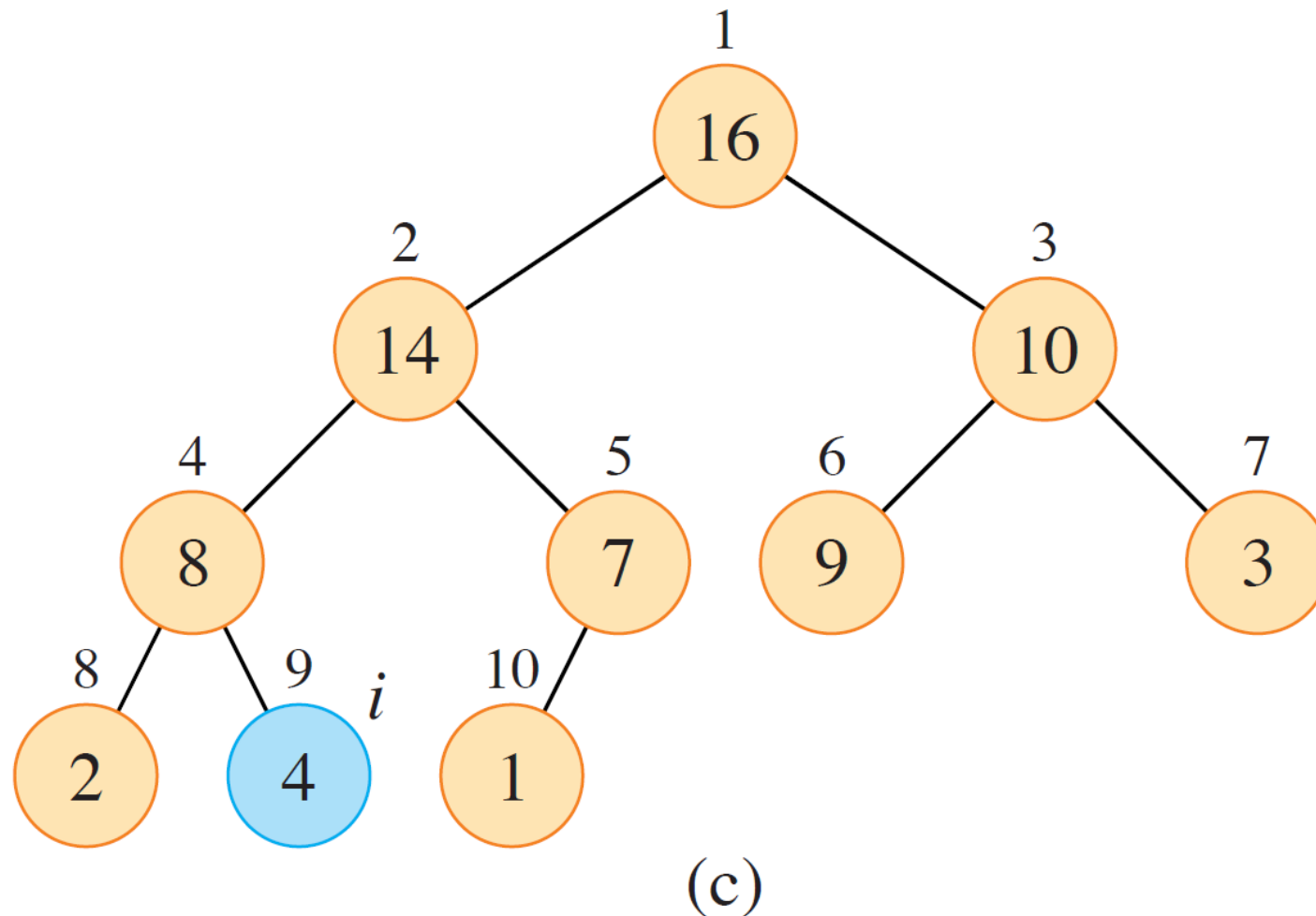


(a)

7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP



7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP



7.3 MANUTENÇÃO DA PROPRIEDADE DE HEAP

```
MAXIMIZA-HEAP(A, i)
1  l = ESQUERDA(i)
2  r = DIREITA(i)
3  if l ≤ A.tamanho-do-heap e A[l] > A[i]
4      maior = l
5  else maior = i
6  if r ≤ A.tamanho-do-heap e A[r] > A[maior]
7      maior = r
8  if maior ≠ i
9      trocar A[i] com A[maior]
10     MAXIMIZA-HEAP(A, maior)
```

7.4 CONSTRUÇÃO DE UM HEAP

- O procedimento CONSTRÓI-MAX-HEAP converte um vetor $A[1 : n]$ em um heap de máximo chamando MAXIMIZA-HEAP de baixo para cima.
- Os elementos no subvetor $A[\lfloor n/2 \rfloor + 1 : n]$ são todos folhas da árvore e, portanto, cada um é um heap que inicia com 1 elemento.
- CONSTRÓI-MAX-HEAP percorre os nós restantes da árvore e executa MAXIMIZA-HEAP sobre cada um.

7.4 CONSTRUÇÃO DE UM HEAP

CONSTRÓI-MAX-HEAP(A, n)

1 A.tamanho-do-heap = n

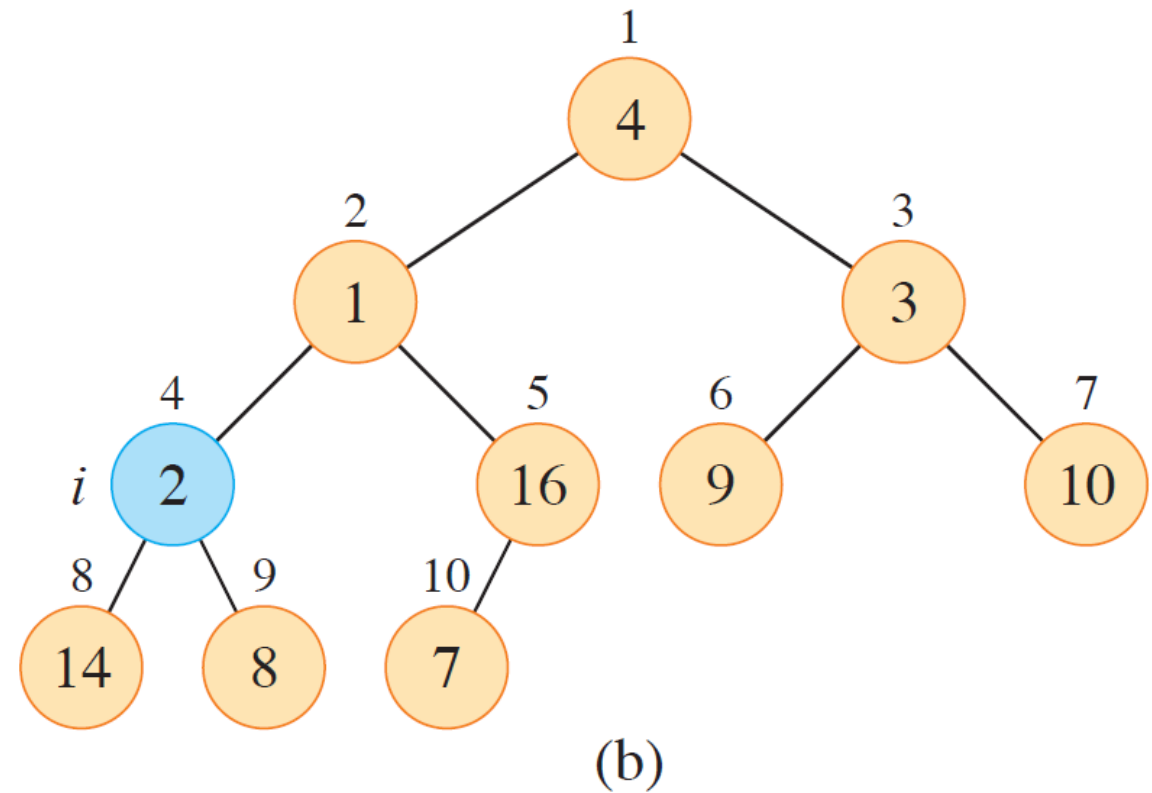
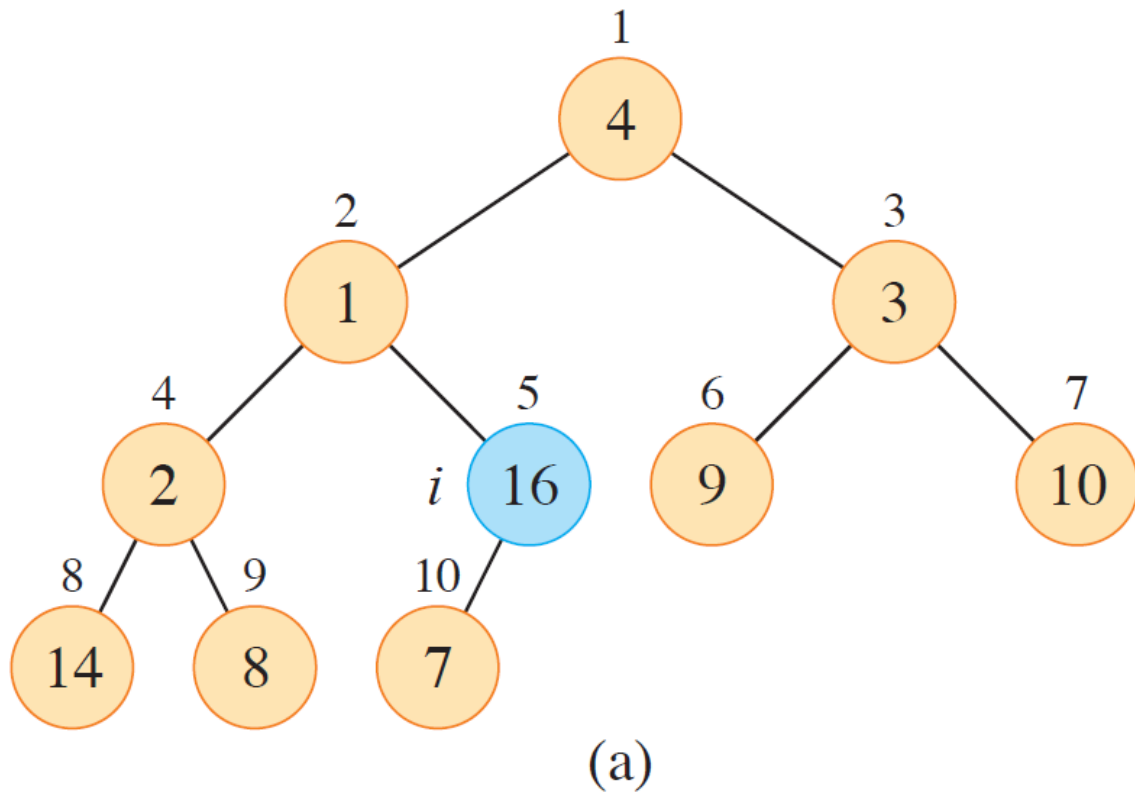
2 for i = $\lfloor n/2 \rfloor$ até 1

3 MAXIMIZA-HEAP(A, i)

7.4 CONSTRUÇÃO DE UM HEAP

A

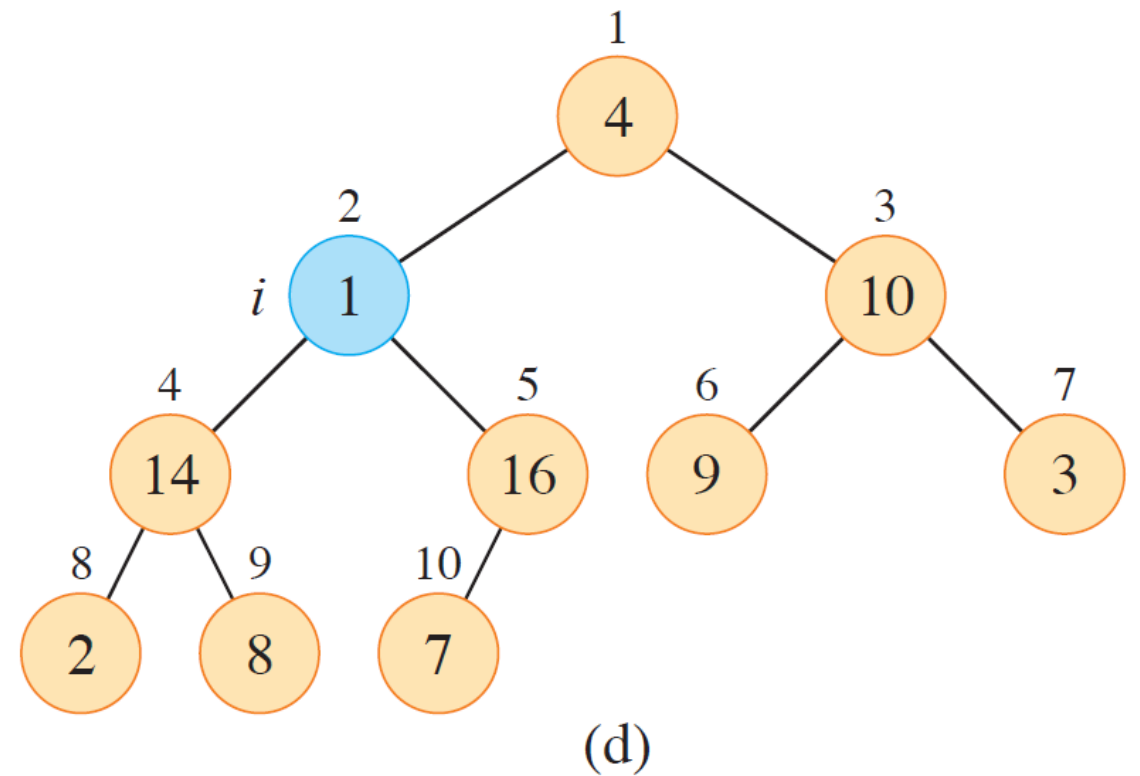
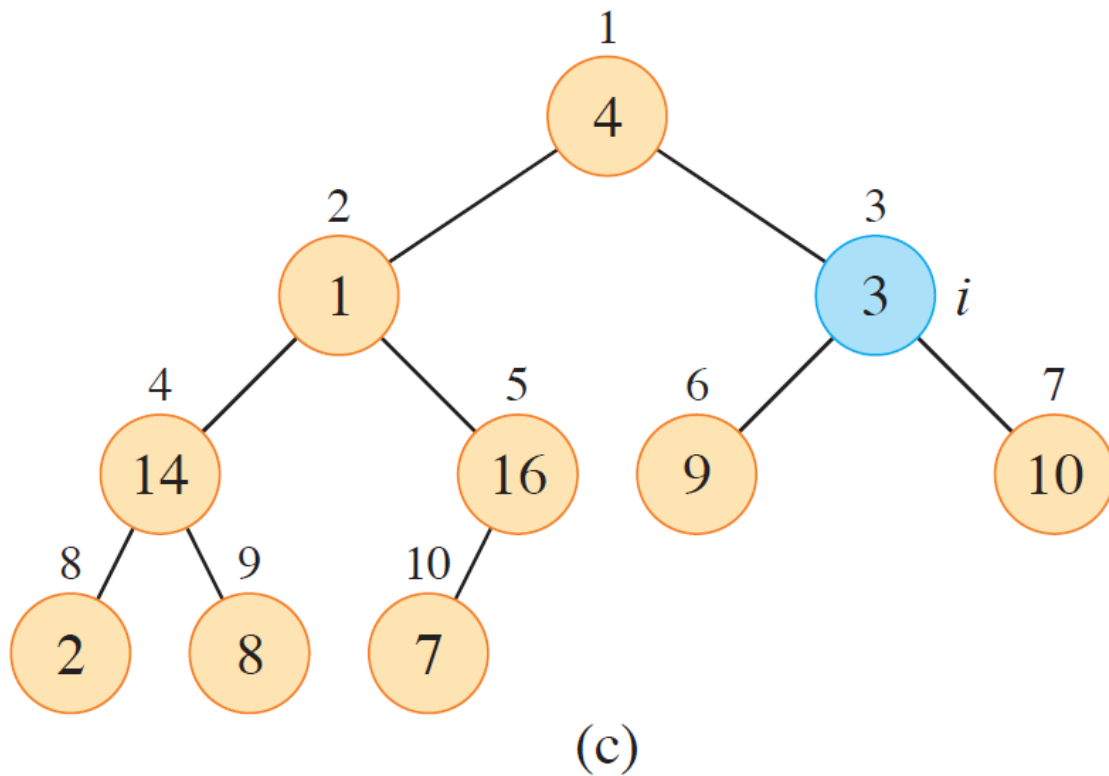
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



7.4 CONSTRUÇÃO DE UM HEAP

A

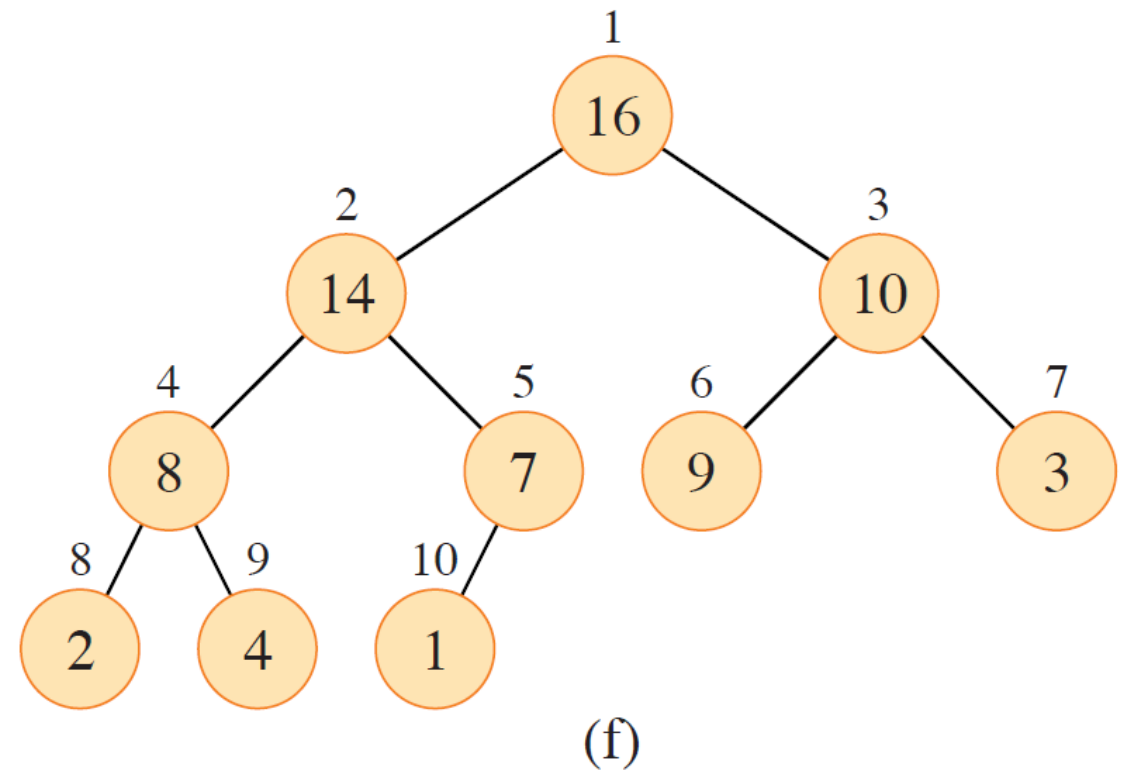
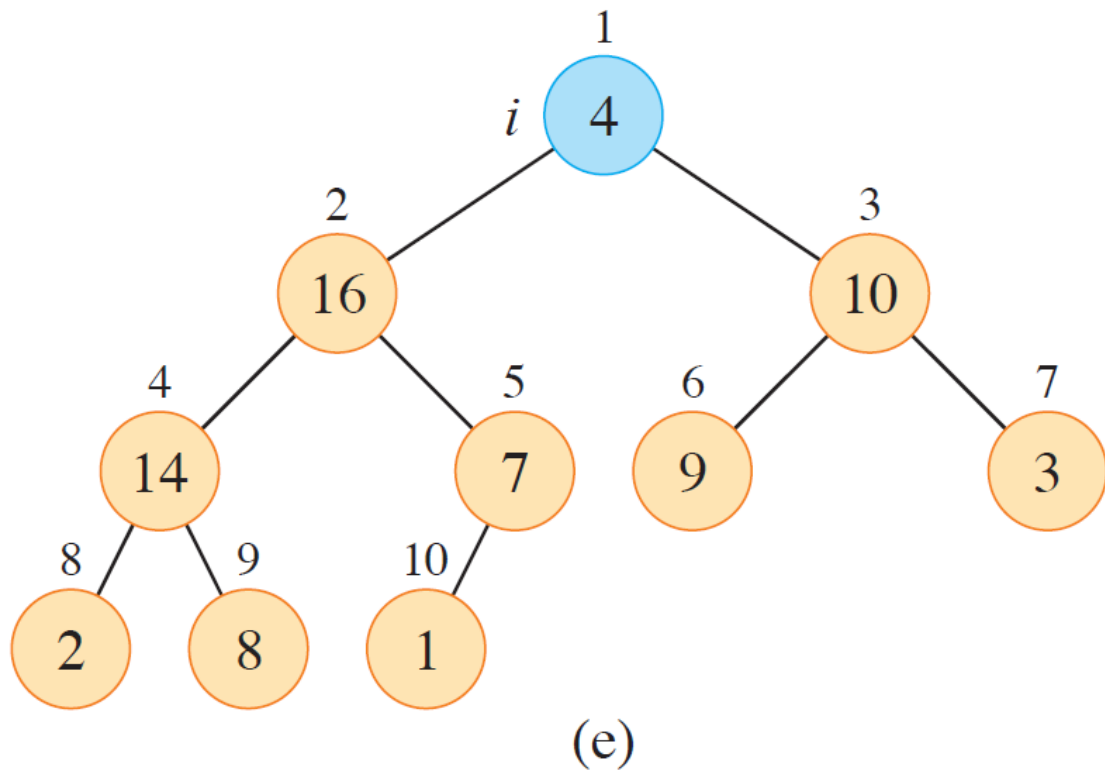
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



7.4 CONSTRUÇÃO DE UM HEAP

A

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



7.4 CONSTRUÇÃO DE UM HEAP

- Vemos que CONSTRÓI-MAX-HEAP funciona corretamente de acordo com o seguinte invariante de laço:

No começo de cada iteração do laço for das linhas 2–3, cada nó $i + 1$, $i + 2$, ..., n é a raiz de um heap de máximo.

- De acordo com um **limite assintótico mais restrito**, podemos construir um heap de máximo a partir de um vetor não ordenado em **tempo linear**.

7.5 HEAPSORT

- O algoritmo de ordenação por heap começa usando CONSTRÓI-MAX-HEAP para construir um heap de máximo no vetor de entrada $A[1 : n]$.
- Visto que o elemento máximo do vetor está armazenado na raiz $A[1]$, HEAPSORT pode colocá-lo em sua posição final correta trocando-o por $A[n]$.
- Se o procedimento em seguida descartar o nó n do heap — e para isso basta simplesmente decrementar $A.tamanho-do-heap$ —, os filhos da raiz continuam sendo heaps de máximo, mas o novo elemento raiz pode violar a propriedade de heap de máximo.

7.5 HEAPSORT

- Porém, para restabelecer a propriedade de heap de máximo, o procedimento só precisa chamar $\text{MAXIMIZA-HEAP}(A, 1)$, que deixa um heap de máximo em $A[1 : n - 1]$.
- Então, o procedimento HEAPSORT repete esse processo para o heap de máximo de tamanho $n - 1$ até um heap de tamanho 2.
- O procedimento HEAPSORT demora o **tempo $O(n \lg n)$** , já que a chamada a CONSTRÓI-MAX-HEAP demora o tempo $O(n)$, e cada uma das $n - 1$ chamadas a MAXIMIZA-HEAP demora o tempo $O(\lg n)$.

7.5 HEAPSORT

HEAPSORT(A, n)

1 CONSTRÓI-MAX-HEAP(A, n)

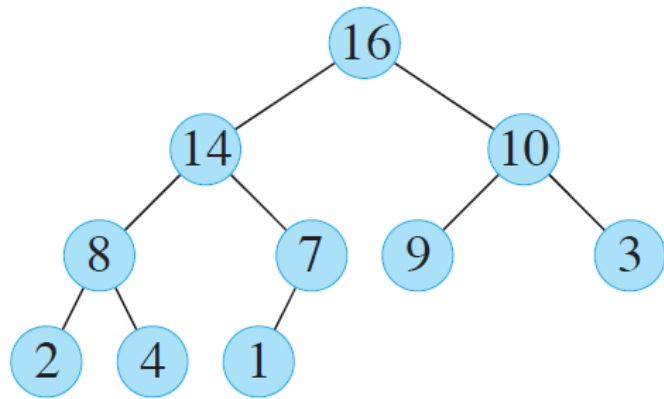
2 for i = n até 2

3 trocar A[1] com A[i]

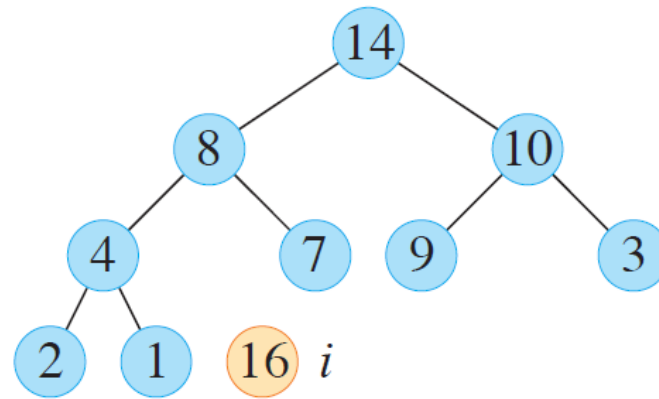
4 A.tamanho-do-heap = A.tamanho-do-heap - 1

5 MAXIMIZA-HEAP(A, 1)

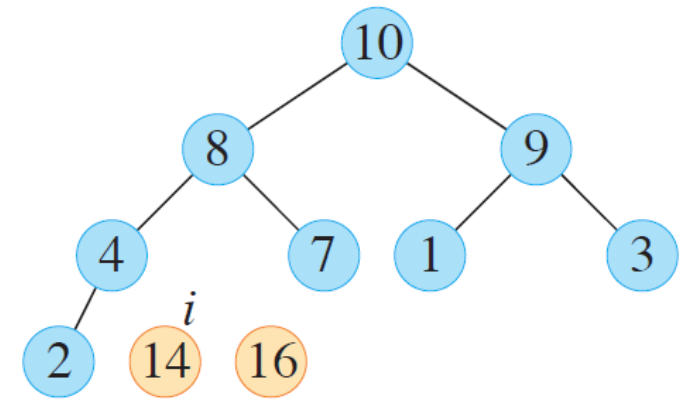
7.5 HEAPSORT



(a)

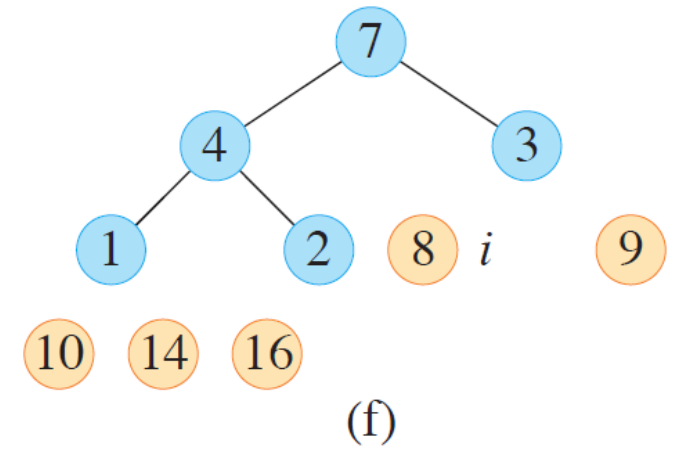
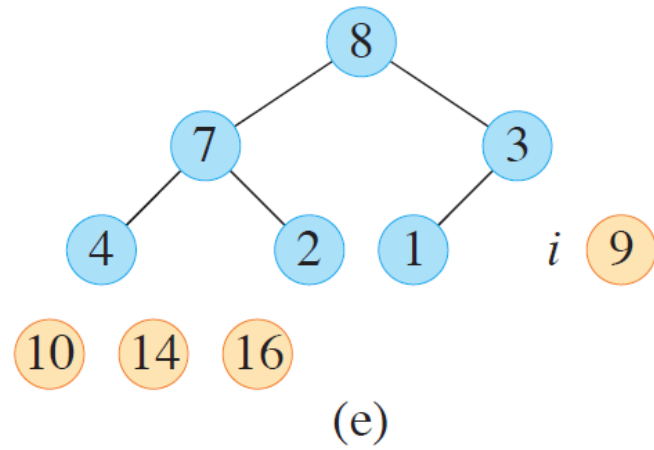
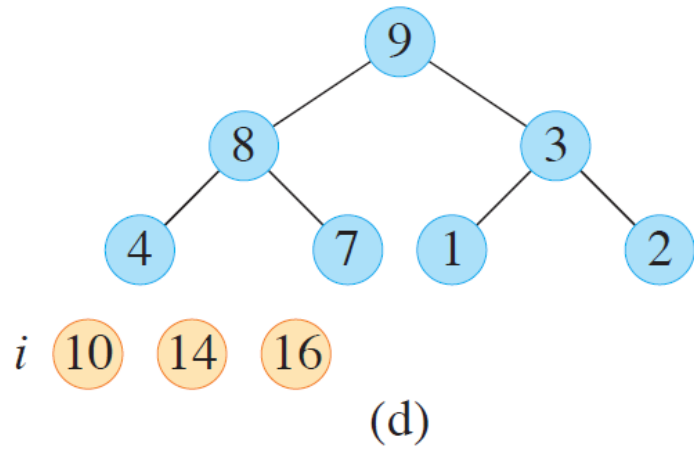


(b)

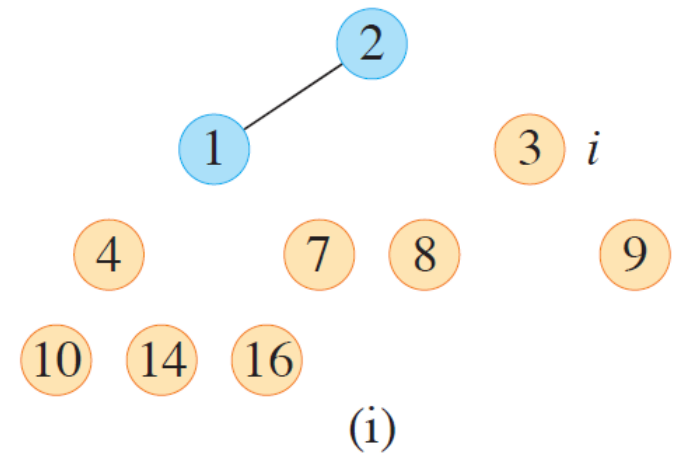
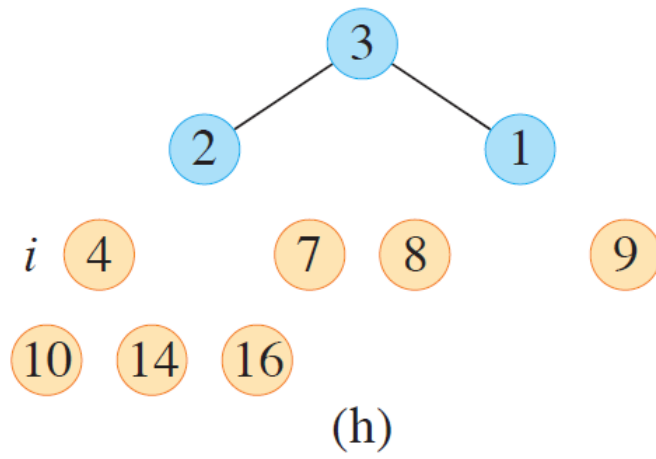
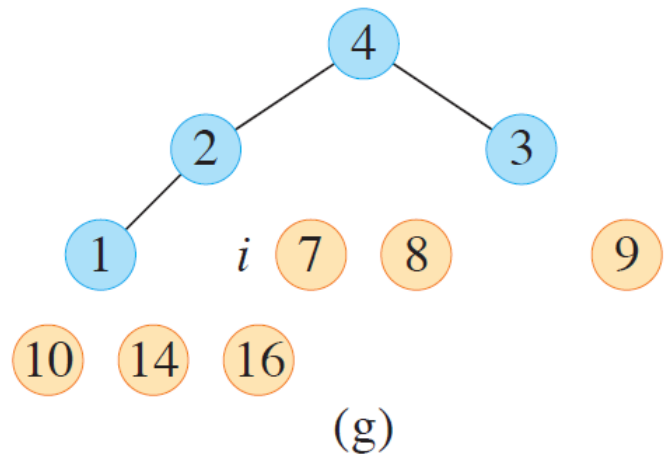


(c)

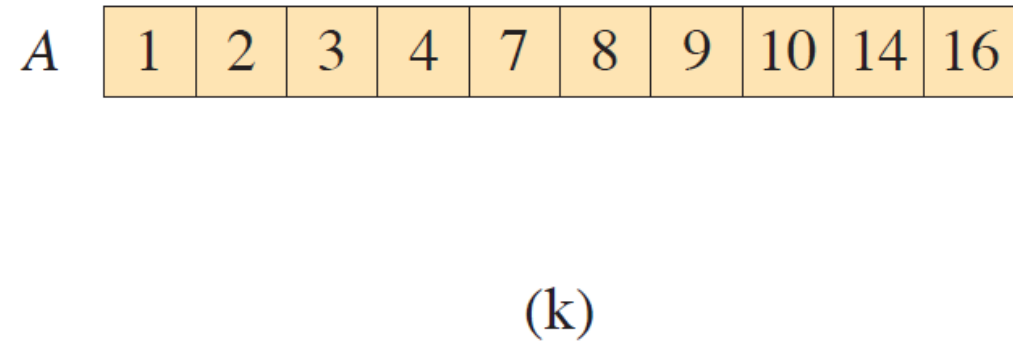
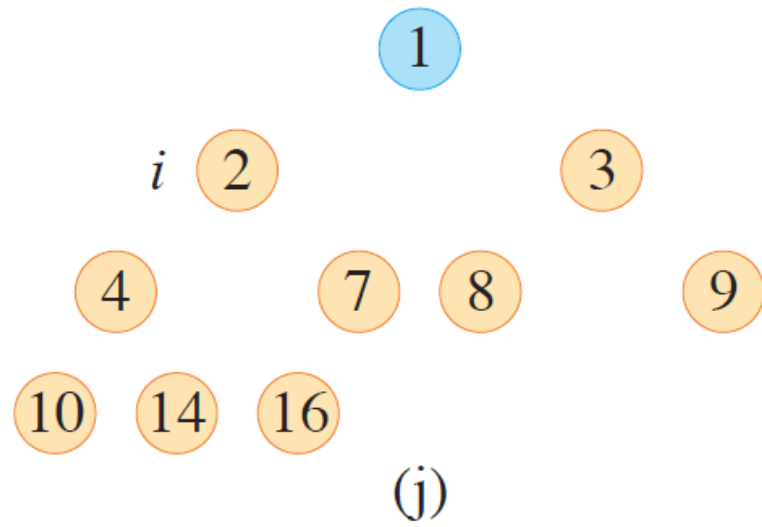
7.5 HEAPSORT



7.5 HEAPSORT



7.5 HEAPSORT



7.6 FILAS DE PRIORIDADE

- A estrutura de dados de heap propriamente dita tem muitas utilidades. Veremos agora como ela pode ser útil na implementação de uma **fila de prioridades eficiente**.
- Como ocorre com os heaps, existem dois tipos de filas de prioridades: filas de prioridade máxima e filas de prioridade mínima.

7.6 FILAS DE PRIORIDADE

- Uma fila de prioridade é uma estrutura de dados para manter um conjunto S de elementos, cada qual com um valor associado denominado chave. Uma fila de prioridade máxima suporta as seguintes operações:
 - **INSERE**(S, x, k) insere o elemento x com chave k no conjunto S , que é equivalente à operação $S = S \cup \{x\}$.
 - **MÁXIMO**(S) retorna o elemento de S que tenha a maior chave.
 - **EXTRAI-MAX**(S) remove e retorna o elemento de S que tenha a maior chave.
 - **AUMENTA-CHAVE**(S, x, k) aumenta o valor da chave do elemento x até o novo valor k , que se admite ser, pelo menos, tão grande quanto o valor da chave atual de x .

7.6 FILAS DE PRIORIDADE

- Entre outras aplicações, podemos usar filas de prioridade máxima para programar trabalhos em um **computador compartilhado por diversos usuários**.
- A fila de prioridade máxima mantém o controle dos trabalhos a executar e suas prioridades relativas.
- Quando um trabalho termina ou é interrompido, o escalonador seleciona o trabalho de prioridade mais alta entre os trabalhos pendentes chamando EXTRAI-MAX. O escalonador pode acrescentar um novo trabalho à fila em qualquer instante chamando INSERE.

7.6 FILAS DE PRIORIDADE

- Alternativamente, uma fila de prioridade mínima suporta as operações INSERE, MÍNIMO, EXTRAI-MIN e DIMINUI-CHAVE. Uma fila de prioridade mínima pode ser usada em um **simulador orientado a eventos**.
- Os itens na fila são eventos a simular, cada qual com um instante de ocorrência associado que serve como sua chave. Os eventos devem ser simulados na ordem de seu instante de ocorrência, porque a simulação de um evento pode provocar outros eventos a simular no futuro.

7.6 FILAS DE PRIORIDADE

- Quando usamos um heap para implementar uma fila de prioridades dentro de determinada aplicação, os elementos na fila de prioridades correspondem a **objetos na aplicação**. Cada objeto contém uma chave.
- Se a fila de prioridades for implementada por um heap, você precisa determinar qual objeto da aplicação corresponde a determinado elemento do heap, e vice-versa.
- Como os elementos do heap são armazenados em um vetor, é preciso encontrar uma maneira de **mapear os objetos** de aplicação de e para índices do vetor.

7.6 FILAS DE PRIORIDADE

- Uma forma de mapear entre os objetos de aplicação e os elementos do heap utiliza **descritores**, que são informações adicionais armazenadas nos objetos e elementos do heap que dão informações suficientes para realizar o mapeamento.
- Os descritores são frequentemente implementados para serem opacos ao código em redor, mantendo, assim, uma barreira de abstração entre a aplicação e a fila de prioridades.

7.6 FILAS DE PRIORIDADE

- Por exemplo, o descritor dentro de um objeto de aplicação pode conter o índice correspondente para o vetor do heap. Porém, como apenas o código para a fila de prioridades acessa esse índice, o índice fica totalmente escondido do código da aplicação.
- Visto que os elementos do heap mudam de local dentro do vetor durante as operações do heap, uma implementação real da fila de prioridades, ao relocar um elemento do heap, também precisa atualizar os índices do vetor nos descritores correspondentes.

7.6 FILAS DE PRIORIDADE

- Por outro lado, cada elemento no heap poderia conter um ponteiro para o objeto correspondente na aplicação, mas o elemento do heap conhece esse ponteiro somente como um descritor opaco, e a aplicação mapeia esse descritor a um objeto de aplicação.
- Normalmente, o custo adicional (overhead) no pior caso para a manutenção de descritores é $O(1)$ por acesso.
- Como alternativa para incorporar descritores em objetos de aplicação, pode-se armazenar na fila de prioridades um mapeamento entre objetos de aplicação e índices de vetor no heap.

7.6 FILAS DE PRIORIDADE

- Quando um heap implementa uma fila de prioridades, tratamos cada elemento do vetor como um ponteiro para um objeto na fila de prioridades, de modo que o objeto seja análogo aos dados satélites durante a ordenação.
- Além disso, assumimos que cada um desses objetos possui uma chave de atributo, que determina o local no heap ao qual o objeto pertence.
- Para um heap implementado por um vetor A , nos referimos a $A[i].chave$.

7.6 FILAS DE PRIORIDADE

- Assumimos implicitamente que MAXIMIZA-HEAP compara os objetos da fila de prioridades com base em seus atributos de chave.
- Assumimos também que, quando MAXIMIZA-HEAP troca os elementos no vetor, ele está trocando ponteiros, e também que ele atualiza o mapeamento entre objetos e índices do vetor.

7.6 FILAS DE PRIORIDADE

MÁXIMO-MAX-HEAP(A)

```
1 if A.tamanho-do-heap < 1  
2     Erro “underflow do heap”  
3 return A[1]
```

7.6 FILAS DE PRIORIDADE

EXTRAÍ-MAX-MAX-HEAP(A)

1 max = MÁXIMO-MAX-HEAP(A)

2 A[1] = A[A.tamanho-do-heap]

3 A.tamanho-do-heap = A.tamanho-do-heap - 1

4 MAXIMIZA-HEAP(A, 1)

5 return max

7.6 FILAS DE PRIORIDADE

- O tempo de execução de EXTRAI-MAX-MAX-HEAP é $O(\lg n)$, pois ele realiza apenas uma quantidade constante de trabalho além do tempo $O(\lg n)$ para MAXIMIZA-HEAP, mais qualquer custo adicional incorrido com MAXIMIZA-HEAP para mapear objetos da fila de prioridades a índices do vetor.

7.6 FILAS DE PRIORIDADE

- **Exercício:** estudar os procedimentos AUMENTA-CHAVE-MAX-HEAP(A, x, k) e INSERE-MAX-HEAP(A, x, n) (Seção 6.5 do Livro Algoritmos Thomas H. Cormen).